Лабораторная работа 4 Эксперименты и базовая оценка модели

Версия: 2026–2027 учебный год

Формат: команда 2–5 человек с индивидуальной сдачей

Предшествующая работа: ЛР3 «Версионирование данных и воспроизводимые вычисления»

1 Цель

Сделать обучение модели наблюдаемым и сравнимым: зарегистрировать не менее двух MLflow runs, связать каждый запуск с Git commit и версией данных, сохранить параметры, метрики, входной набор и артефакт pipeline, а затем обосновать выбор базовой конфигурации только по validation-периоду.

2 Результаты обучения

Каждый студент должен уметь:

- различать experiment, run, parameter, metric, tag, input и artifact;
- запускать локальный MLflow Tracking Server или согласованный преподавателем файловый backend;
- логировать параметры модели и временного разбиения;
- связать run с Git commit, DVC hash, SHA-256 и версией контракта;
- регистрировать training и validation datasets как входы;
- сравнивать не менее двух запусков по одной и той же методике;
- объяснять различие validation и test и не выбирать модель по test;
- загрузить сохранённый pipeline-артефакт и повторить предсказание.

3 Командный результат

Команда предоставляет:

1. experiment с названием по шаблону преподавателя;
2. не менее двух завершённых MLflow runs с различающимся гиперпараметром или алгоритмом;
3. для каждого run: параметры, validation-метрики, dataset inputs, теги версий и модельный артефакт;
4. единый отчёт сравнения `reports/experiments/lr4-comparison.md`;
5. JSON-артефакт с границами train, validation и test и числами строк;
6. проверку загрузки модели из артефакта и повторного предсказания;
7. issue, pull request и содержательное review;
8. отдельную evidence card каждого участника;
9. tag `lr4-v1` после успешных проверок.

4 Ограничение оценки

В ЛР4 модель выбирается только по validation-периоду. Test сохраняется закрытым для итоговой проверки и не используется для перебора параметров. Основная метрика учебного бинарного классификатора — ROC AUC на validation; дополнительно фиксируются precision, recall, F1 и доля положительного класса. Если в части данных есть только один класс, run завершается понятной ошибкой, а не фиктивной метрикой.

Небольшой синтетический набор стартового проекта предназначен для smoke-проверки. Содержательное сравнение выполняется на подготовленном преподавателем временном фрагменте данных.

5 Роли команды

Роль	Ответственность	Индивидуальное доказательство
Experiment owner	схема experiment и обязательных tags	run и описание соглашения
Model engineer	две конфигурации обучения	diff параметров и код pipeline
Evaluation engineer	метрики и границы частей	тест метрик и comparison report
Reproducibility reviewer	Git/DVC/data linkage	проверка tags и восстановление run
Quality and documentation reviewer	PR, загрузка модели, evidence	содержательное review и smoke test

При меньшем составе роли совмещаются. Роль должна отличаться от двух предыдущих ЛР, если это возможно.

6 Подготовка окружения

Создайте ветку и установите дополнительный набор зависимостей:

```
git switch main
git pull --ff-only
git switch -c lab4/mlflow-tracking
python -m pip install -e ".[dev,tracking]"
python -m pytest -q
```

Зафиксируйте исходное состояние:

```
git rev-parse HEAD
dvc status
python -m mlops_flight_delay.validate_data --data data/flights_sample.csv --contract data/contract.json
```

Рабочее дерево перед запуском должно быть чистым либо run получает tag `working_tree_dirty=true` с объяснением. Для принимаемой версии рекомендуется чистое состояние.

7 Локальный Tracking Server

Для командной работы используйте локальную SQLite-базу и каталог артефактов внутри разрешённого учебного окружения:

```
mlflow server --host 127.0.0.1 --port 5000
```

Во втором терминале задайте tracking URI:

```
$env:MLFLOW_TRACKING_URI="http://127.0.0.1:5000"
```

В Linux или macOS используйте `export`. Не публикуйте MLflow server во внешнюю сеть без аутентификации, ограничения хостов и согласования с преподавателем. Папки `mlruns`, `mlartifacts` и локальная SQLite-база не добавляются в Git.

8 Соглашение об experiment и run

Название experiment: `mlops-<group>-<team>-flight-delay`. Run name содержит короткую конфигурацию, например `rf-depth-3`.

Обязательные tags:

Tag	Содержание
`git.commit`	полный Git commit SHA
`git.dirty`	`true` или `false`
`data.dvc_hash`	hash версии данных из ЛР3
`data.sha256`	SHA-256 CSV
`data.contract_version`	версия контракта
`split.policy`	`chronological-60-20-20` или утверждённые даты

Tag	Содержание
`team.code`	код команды без персональных данных
`validation.status`	`candidate`, `rejected` или `invalid`

Не используйте tag `Production` или stage как решение о выпуске. В ЛР4 Registry ещё не применяется; управляемый выпуск относится к ЛР10.

9 Что логировать

Параметры должны включать алгоритм, `n_estimators`, `max_depth`, `random_state`, порог целевого класса и границы частей. Метрики должны иметь явный контекст, например `validation_roc_auc`, а не просто `score`.

Для dataset inputs используйте API MLflow для pandas-наборов:

```
training_input = mlflow.data.from_pandas(
train_frame,
source="data/flights_sample.csv",
name="flight-delay-train",
targets="target",
)
mlflow.log_input(training_input, context="training")
```

Не логируйте полный закрытый набор как artifact. Dataset input должен сохранять разрешённые метаданные, digest, схему и источник. Политика курса и лицензия данных имеют приоритет над удобством интерфейса.

10 Два обязательных запуска

Выполните две конфигурации при неизменных данных и границах:

```
python -m mlops_flight_delay.train_mlflow --data data/flights_sample.csv --contract data/contract.json --n-estimators
80 --max-depth 3 --experiment mlops-<group>-<team>-flight-delay
python -m mlops_flight_delay.train_mlflow --data data/flights_sample.csv --contract data/contract.json --n-estimators
160 --max-depth 6 --experiment mlops-<group>-<team>-flight-delay
```

Изменение сразу нескольких факторов допустимо только при явно сформулированной гипотезе. Для учебного сравнения лучше менять один параметр, чтобы вывод был интерпретируем.

11 Артефакты run

Каждый run сохраняет:

- сериализованный pipeline или model bundle;
- `split_manifest.json` с временными границами;
- `metrics.json` с validation-метриками;
- validation report ЛР2;
- краткий `run_summary.md` с гипотезой и выводом;
- окружение или список зафиксированных зависимостей.

После завершения загрузите модельный артефакт через MLflow API или скачайте artifact в чистый временный каталог. Выполните предсказание на разрешённой строке validation и сравните результат с записанным smoke test.

12 Отчёт сравнения

В `lr4-comparison.md` включите таблицу:

Run ID	Конфигурация	Версия данных	Validation ROC AUC	Precision	Recall	F1	Решение
...	RF depth 3	...	...	...	...	...	candidate/rejected

Run ID	Конфигурация	Версия данных	Validation ROC AUC	Precision	Recall	F1	Решение
...	RF depth 6	...	...	...	...	...	candidate/rejected

После таблицы сформулируйте гипотезу, наблюдение, ограничение выборки и решение. Разница метрик на маленьком наборе не доказывает превосходство модели. Если значения одинаковы, это тоже результат, требующий объяснения.

13 Автоматические проверки

Добавьте тесты, которые подтверждают:

1. хронологическое разбиение и отсутствие пересечения частей;
2. наличие обоих классов перед вычислением ROC AUC;
3. сохранение обязательных полей model bundle;
4. загрузку артефакта и форму предсказания;
5. отказ при неверном контракте или невозможной метрике.

MLflow server не должен быть обязательным для всех unit-тестов. Отделите чистые функции обучения и оценки от интеграционного теста tracking.

14 Индивидуальная сдача участника команды

Командный результат: tag `lr4-v1`, два run ID и отчёт сравнения.

Каждый участник до защиты размещает индивидуальную evidence card. Совпадающие карточки не принимаются.

Поле	Что указать
Студент	ФИО, группа, команда, GitHub-логин
Роль	Роль в ЛР4 и отличие от роли в ЛР3
Задача	issue с личным критерием приёмки
Личный артефакт	PR, commit, review, тест, run ID или отчёт
Вклад	конкретные созданные или проверенные элементы
Инженерное решение	выбранный параметр, метрика или способ логирования и альтернатива
Риск	утечка test, несвязанная версия данных, неполный run или закрытые данные
Воспроизведение	run ID, Git SHA, DVC hash и команда загрузки артефакта
Использование ИИ	инструмент и самостоятельно проверенные утверждения либо отсутствие использования

Минимальный допуск: командный tag существует, личная issue закрыта проверяемым результатом, PR прошёл review другого участника, оба runs завершены, обязательные tags и artifacts доступны.

15 Порядок индивидуальной защиты

За 3–4 минуты студент:

1. открывает назначенный преподавателем run ID;
2. показывает связь с Git commit и DVC hash;
3. объясняет один параметр, одну метрику и один artifact;

4. загружает модельный артефакт или запускает smoke test;
5. сравнивает run с альтернативной конфигурацией;
6. объясняет, почему test не использовался для выбора.

Преподаватель может попросить найти run по tag, изменить один параметр, показать неуспешный run или объяснить расхождение метрики.

16 Повторная сдача и стоп-факторы

При повторной сдаче создаётся или переоткрывается issue, изменения вносятся новым PR, создаётся новый run, а история неуспешной попытки сохраняется.

Стоп-факторы:

- нет личного артефакта или студент не может объяснить заявленный run;
- run не связан с Git и данными;
- модель выбрана по test-периоду;
- закрытые данные или секреты опубликованы как artifacts;
- метрика скопирована вручную и не воспроизводится;
- один run выдан за сравнение двух конфигураций.

17 Критерии оценивания

Критерий	Баллы	Полное выполнение
Структура experiment и runs	2	два сопоставимых завершённых запуска
Прослеживаемость	2	Git, DVC, SHA-256, контракт и split связаны tags
Метрики и входы	2	корректные validation-метрики и dataset inputs
Артефакты	1	модель, split manifest, отчёт и повторная загрузка
Командный процесс	1	issue, PR и содержательное review
Индивидуальная защита	2	личный вклад, воспроизведение и объяснение риска

Максимум: 10 баллов. Рекомендуемый порог зачёта: 6 баллов при отсутствии стоп-факторов.

18 Чек-лист команды

- [ ] Состав команды 2–5 человек, роли ротированы.
- [ ] Данные и контракт соответствуют принятой ЛР3.
- [ ] Созданы два run с различающимися конфигурациями.
- [ ] Каждый run связан с Git commit и DVC hash.
- [ ] Логируются параметры, validation-метрики и dataset inputs.
- [ ] Test не используется для выбора.
- [ ] Модельный артефакт загружается и выполняет предсказание.
- [ ] Отчёт содержит сравнение и ограничение выводов.
- [ ] PR прошёл независимое review.
- [ ] Каждый участник подготовил evidence card и индивидуальную демонстрацию.

19 Связь с ЛР5

В ЛР5 зафиксированные функции данных, обучения и оценки будут покрыты систематическими тестами и fixtures. Выбранный run остаётся базовой линией, но не переводится в Registry до управляемого выпуска в ЛР10.

20 Официальные источники

- MLflow Tracking: https://mlflow.org/docs/latest/ml/tracking/
- MLflow Tracking Quickstart: https://mlflow.org/docs/latest/ml/getting-started/quickstart/
- MLflow Dataset Tracking: https://mlflow.org/docs/latest/dataset/